Detailed Chat Application HLD Handbook (SDE-1)

Introduction

This handbook explains the complete high-level design of a scalable chat application and the reasoning behind every design choice from an SDE-1 interview perspective.

Requirements

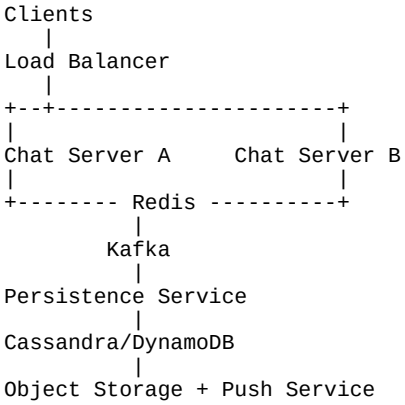
Functional: Login, one-to-one messaging, group chat, history, presence, typing indicator, read receipts, offline synchronization, notifications and media.

Non-functional: Low latency, high availability, scalability, durability, reliability, security and fault tolerance.

Capacity Estimation

Example assumptions: 100M daily active users, 100 messages/day/user, around 10B messages/day, approximately 115K average QPS and 250K peak QPS. Around 150 bytes per message gives roughly 1.5TB/day storage.

Architecture



Connection Lifecycle

Authenticate user, establish WebSocket, store UserID→ServerID in Redis, refresh via heartbeats, remove mapping on disconnect.

Message Flow

Sender -> Chat Server -> Redis lookup -> Destination Chat Server -> Receiver. Publish to Kafka and persist asynchronously.

Offline Flow

Persist message, trigger push notification, fetch missed messages after reconnect.

Presence

Use Redis TTL and heartbeats because presence changes frequently.

Typing

Ephemeral WebSocket event, not persisted.

Read Receipts

Receiver sends read event, sender is notified asynchronously.

Database

Partition by conversation ID. Store IDs, timestamps, status, payload and media URL.

Kafka

Provides buffering, durability, retries and multiple consumers.

Ordering

Partition by conversation and attach sequence numbers.

Idempotency

Use UUID message IDs to prevent duplicate processing.

Group Chat

Fan-out-on-write for small groups, hybrid/fan-out-on-read for large groups.

Media

Store files in object storage and save URLs in database.

Scaling

Stateless servers, Redis Cluster, Kafka partitions, sharding, replication, multi-region deployment.

Failure Handling

Replicate Redis/Kafka/DB. Retry failed deliveries. Recover using durable queues.

Security

TLS, authentication, authorization, optional end-to-end encryption.

Interview Strategy

Clarify requirements, estimate scale, build incrementally, justify every component and discuss trade-offs.

Interview Questions

Q: Why WebSockets?
A: Persistent bidirectional communication with minimal latency.

Q: Why Redis?
A: Fast presence and routing lookups.

Q: Why Kafka?
A: Durable asynchronous messaging.

Q: Why Cassandra?
A: High write throughput and horizontal scaling.

Q: How avoid message loss?
A: Publish to durable queue before persistence.

Q: How preserve ordering?
A: Conversation-based partitioning.

Q: How deliver offline messages?
A: Persist then synchronize on reconnect.

Q: Why not store typing?
A: It is temporary state.

Q: How prevent duplicates?
A: Idempotency using UUIDs.

Q: How scale?
A: Stateless services, sharding, partitioning and replication.

Revision Sheet

WebSockets → Real-time messaging | Redis → Presence & routing | Kafka → Durable queue | Cassandra/DynamoDB → Storage | Object Storage → Media | Push Notifications → Offline users | UUID → Idempotency | Conversation Partition → Ordering | Heartbeats → Presence | Sharding + Replication → Scale.